

¿Por qué tu código parece un laberinto?

El coste real de ignorar el "sentido común" en las APIs

Llevas 20 minutos mirando la documentación de una API que debería ser sencilla. Los endpoints, las direcciones a través de las cuales dos programas se comunican, existen, los parámetros están ahí, y aun así algo no cuadra. No es que no sepas lo que haces, es que quien diseñó eso tomó decisiones que tu cerebro tiene que deshacer antes de poder avanzar. Un [estudio](#) publicado en 2023 con 105 desarrolladores reales pone número a ese tiempo perdido, y los resultados deberían cambiar cómo cualquier equipo revisa su código.

El experimento que nadie había hecho

Investigadores de universidades de Alemania y los Países Bajos montaron algo aparentemente simple. Cogieron 12 reglas básicas de diseño de APIs, cosas como cómo nombrar los recursos o qué método HTTP usar para cada acción, y crearon dos versiones de cada fragmento de código. Una seguía la regla. La otra la violaba. Luego pusieron a 105 personas, entre estudiantes y profesionales con años de experiencia, a resolver tareas de comprensión sobre cada versión.

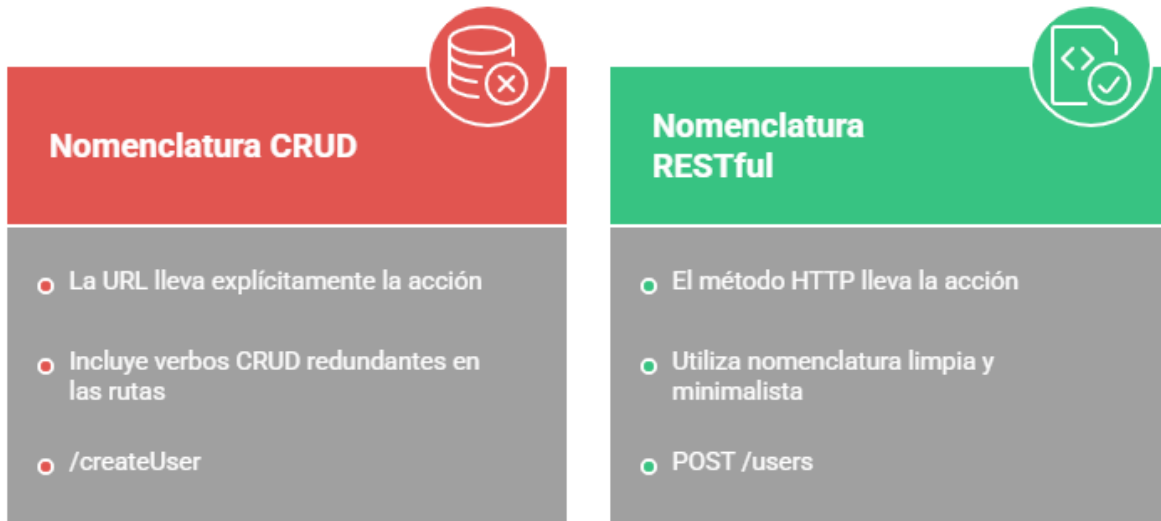
La métrica principal que usaron se llama **TAU**, tiempo ajustado de comprensión real. Es como si en lugar de medir solo si un corredor llega a la meta, midieras cuánto esfuerzo le costó el camino. No solo si acertabas, sino cuánto tardaba tu cerebro en llegar ahí. En 11 de las 12 reglas, la versión que violaba la norma salió perdiendo de forma significativa.

El error más caro: mezclar acciones con direcciones

La regla con mayor impacto en todo el experimento fue la de los **nombres CRUD en la URL**. CRUD es el acrónimo de las cuatro operaciones básicas sobre datos: crear, leer, actualizar y borrar. Poner esas palabras, create, delete, update, dentro de la dirección de un endpoint fue con diferencia lo que más confundió a los participantes. El motivo es simple: el método HTTP ya cumple esa función, es la instrucción que indica qué operación se realiza.

Tiene lógica si lo piensas. Una dirección web debería decirte *qué* estás tocando, no *qué vas a hacerle*. Imagina que entras a una tienda y en lugar de ver una sección llamada "Zapatos" ves un cartel que dice "Comprar zapatos": la información está ahí, pero tu cerebro tiene que hacer un paso extra que no debería ser necesario. Cuando el nombre de la URL ya lleva la acción incorporada, pasa exactamente eso. El estudio lo confirma: el tamaño del efecto de esta regla ($d=2.17$) fue el mayor de todos los analizados, lo que en estadística indica una diferencia práctica muy notable, no solo significativa en el papel.

¿Dónde va la acción?



Made with Napkin

El árbol al revés

La jerarquía de rutas fue otro punto conflictivo. La lógica natural es ir de lo general a lo específico, primero el recurso padre y luego el hijo. Cuando esa jerarquía se invierte o se rompe, el desarrollador tiene que reconstruir mentalmente la relación entre recursos antes de poder hacer nada. No es que no pueda, es que tarda más y se equivoca más por el camino.

Algo parecido pasa con el uso del **singular** cuando debería ir el plural. `/member` en lugar de `/members` parece una tontería, pero el experimento muestra que ralentiza la comprensión de forma consistente. El motivo es el mismo: el cerebro espera un patrón y encuentra otro, y ese desajuste tiene un coste real aunque pequeño.

Rutas Lógicas: El Poder del Diseño Intuitivo



La experiencia no te salva

La parte que más me llamó la atención cuando leí el estudio es que el efecto se mantuvo independientemente de la experiencia de los participantes. Da igual si llevas 2 años o 12, ante una API mal diseñada, todos tardan más y todos se equivocan más. Ni siquiera conocer modelos teóricos avanzados compensó un mal diseño.

Eso tiene una implicación directa para cualquier equipo: **no puedes resolver con experiencia lo que está roto en el diseño.**

Diseñar una API es un acto de comunicación

Lo que el estudio demuestra, en el fondo, no es solo que hay reglas que conviene seguir. Es que el diseño de una API no es un trámite técnico que haces antes de ponerte a escribir código de verdad. Es la primera decisión que afecta a todos los que van a usar tu trabajo, incluido tú mismo dentro de unos meses cuando ya no recuerdes por qué hiciste las cosas así.

La próxima vez que diseñes un endpoint, la pregunta no es solo si funciona. Es si alguien que no estuvo en esa reunión puede entenderlo sin tener que preguntarte.